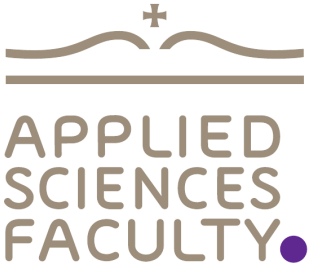
C++ — додаток. Qt. Lecture XX — Бази даних





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- База даних (БД, DB) — організована колекція даних.
- Системи керування базами даних (Database management systems — DBMS) — програмне забезпечення, призначене для роботи із базами даних. Іноді, для простоти, його теж називають базами даних, однак слід розуміти, що це – жаргон.
- Популярні системи керування БД: MySQL, PostgreSQL, Microsoft SQL Server, Oracle, SAP, IBM DB2.
- В загальному випадку, бази даних не є переносимими між різними DBMS, однак взаємодію між ними можна налагодити, за допомогою стандартів, таких як SQL, ODBC, JDBC.
- DBMS класифікують за моделями баз даних, які вони підтримують. Найбільшою популярністю останні десятиліття користуються так звані реляційні бази даних, для роботи з якими служить SQL.
- Останнім часом, для особливо великих чи особливо вимогливих до продуктивності баз даних використовують різноманітні архітектури, зібрані під словом "noSQL".

Посилання

Бази даних – величезна область комп'ютерних наук. Детально ми їх не розглядатимемо. Для початкового ознайомлення див., наприклад:

- `http://en.wikipedia.org/wiki/Database`
- `http://uk.wikipedia.org/wiki/База_даних`
- `http://en.wikipedia.org/wiki/Relational_model`
- `http://uk.wikipedia.org/wiki/Реляційна_модель_даних`
- `http://en.wikipedia.org/wiki/SQL`
- `http://uk.wikipedia.org/wiki/SQL`
- `http://en.wikipedia.org/wiki/Database_transaction`
- `http://uk.wikipedia.org/wiki/Транзакція_(бази_даних)`
- `http://www.bogotobogo.com/VisualBasicSQL/VBLangDatabase1.php` — "Visual Basic 2010 / SQL - Database I (2015)", загальна теорія. (І наступні частини там).
- `http://www.bogotobogo.com/php/php18_mysql_database.php` — "PHP & MySQL Tutorial: MySQL with PHP", загальна теорія.

Операції з БД

Операції над базами даних можна поділити на чотири основних групи:

- Визначення даних (Data definition) — створення, модифікація, знищення визначень, що описують організацію даних.
- Оновлення (Update) — вставка, модифікація, видалення безпосередньо даних.
- Вибірка даних (Retrieval) — отримання даних, як для подальшої обробки, так і для безпосереднього використання.
- Адміністрування (Administration) — керування базами, користувачами, правами доступу, тощо.

Важливим поняттям роботи із БД є транзакції (transactions). Транзакція (wiki) —— група послідовних операцій з базою даних, яка є логічною одиницею роботи з даними. Транзакція може бути виконана або цілком і успішно, дотримуючись цілістності даних і незалежно від інших транзакцій, що йдуть паралельно, або не виконана зовсім, і тоді вона не може справити ніякого ефекту.

Коротко про реляційні ДБ

- Теоретичною основою реляційних баз даних служать реляційні алгебри.
- Дані в реляційних БД зберігаються в **таблицях** (tables). Таблиця має своє ім'я, кількість стопців та кількість рядків у ній.
- Таблиці можуть бути пов'язані між собою.
- Кожен стовпець в таблиці має унікальне ім'я, містить свої дані – той же елемент для різних рядків. Кожен стовпець має свій тип — тип елементів у ньому.
- Рядки – окремі записи у таблиці, один "квант" даних.
- Приклад – рядок, що описує одного клієнта, та містять поля, що відповідають стовпцям імені, прізвища, типових вподобань, тощо.
- Кожен елемент даних (який належить певному рядку та стовпцю) власне зберігає інформацію.
- Ключ — унікальний ідентифікатор запису, рядка.
- Schema (схема? структура?) бази даних — "креслення" БД, всі таблиці, їх стовпці, ключі, тощо, що повністю описують її структуру.
- Зовнішні ключі (Foreign key) забезпечують зв'язок між таблицями. Зв'язок буває трьох видів: один до одного – одному рядку однієї таблиці відповідає один рядок іншої, один до багатьох — коли одному рядку першої таблиці відповідає багато рядків другої і багато до багатьох — коли багато рядків однієї

SQL

- SQL це мова структурованих запитів (Structured query language).
- Вона є декларативною. Тобто, можна описати, що ви хочете зробити із даними, але вказати, як саме це робити, не можна.
- Застосовується для формування запитів, оновлення і керування реляційними БД, створення схеми бази даних і її модифікації, системи контролю за доступом до бази даних[1].
- Сам по собі SQL не є ні системою керування базами даних, ні окремим програмним продуктом. Не будучи мовою програмування в тому розумінні, як C або Pascal, SQL може формувати інтерактивні запити або, будучи вбудованою в прикладні програми, виступати в якості інструкцій для керування даними. Стандарт SQL, крім того, вміщує функції для визначення зміни, перевірки і захисту даних.

[1] Цей і наступний пункт взято, із модифікаціями, з української вікіпедії.

Основні елементи SQL[2]

- DDL (Data Definition Language) — робота із структурою бази,
- DML (Data Manipulation Language) — робота із рядками БД,
- DCL (Data Control Language) — робота з правами,
- TCL (Transaction Control Language) — робота з транзакціями.

[2] За мотивами вікіпедії.

Data Definition Language

- CREATE — створити новий об'єкт (базу, таблицю, індекс, тощо)
- ALTER — зміна об'єкта (наприклад, додавання/зміна полів таблиці)
- DROP — видалення об'єкта

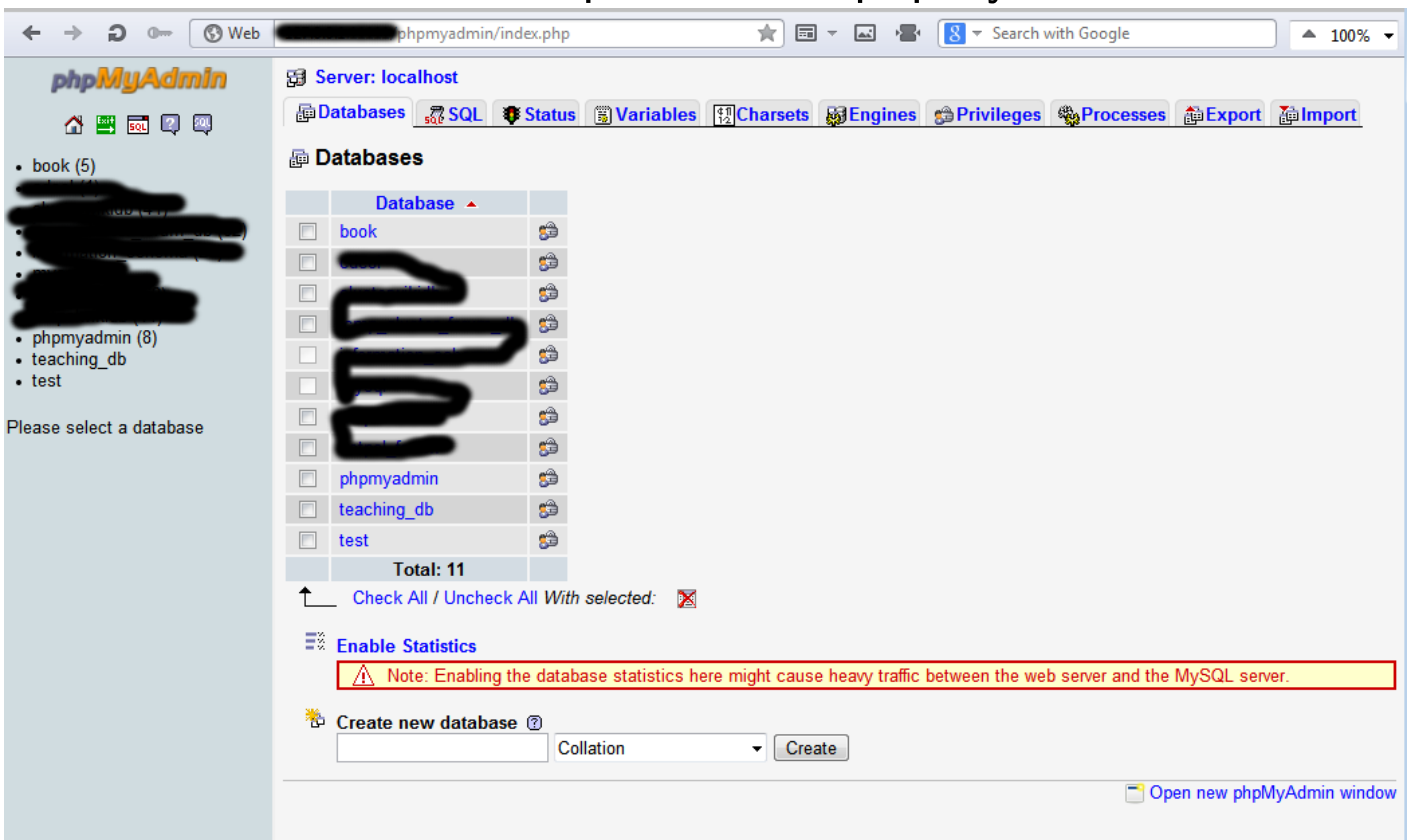
MySQL + phpMyAdmin

CREATE DATABASE teaching_db

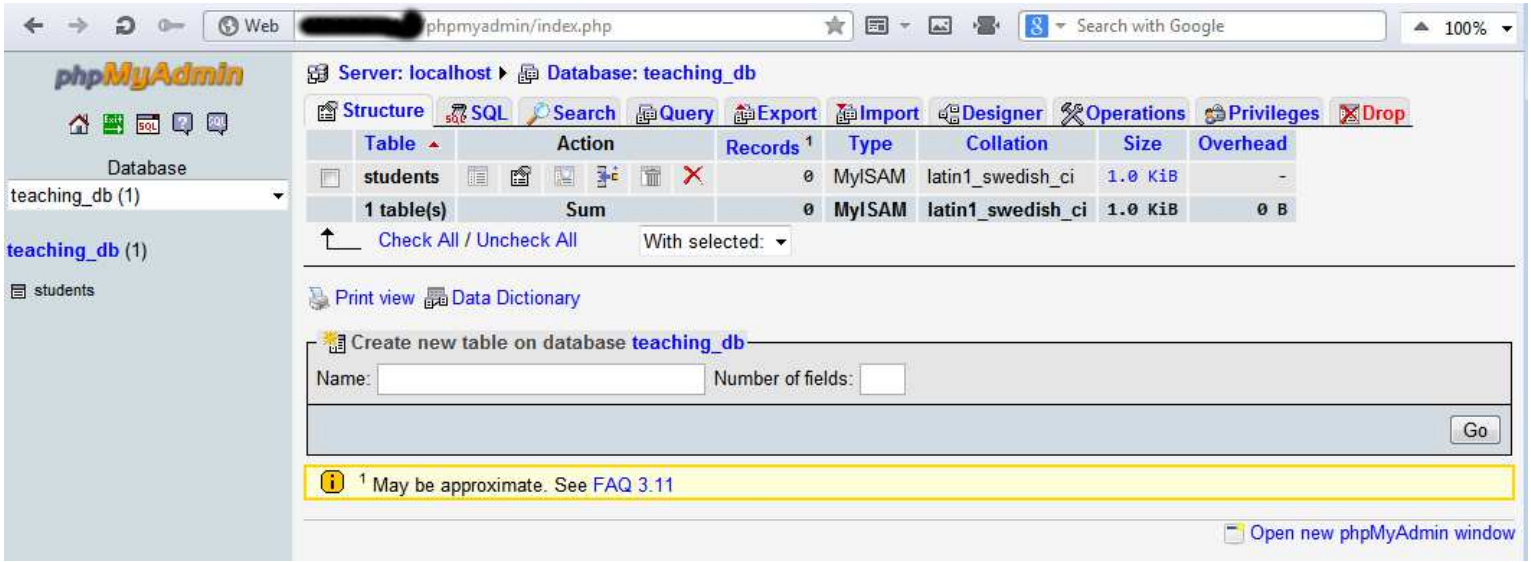
Варіант:

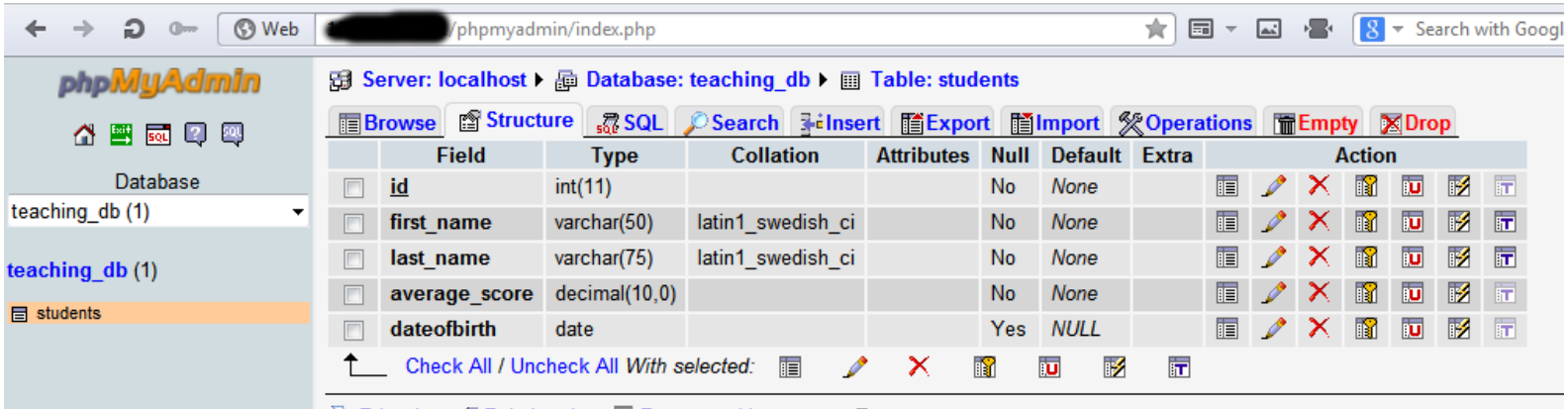
CREATE DATABASE IF **NOT EXISTS** teaching_db

Після чого отримаємо в phpMyAdmin:



```
CREATE TABLE students(
    id  INTEGER PRIMARY KEY NOT NULL AUTO_INCREMENT ,
    first_name VARCHAR( 50 ) NOT NULL ,
    last_name VARCHAR( 75 ) NOT NULL ,
    average_score NUMERIC NOT NULL ,
     dateofbirth  DATE NULL );
```





ALTER TABLE students **ADD** homeworks **INTEGER NOT NULL** ;

TRUNCATE **TABLE** students;

DROP TABLE students;

- ALTER змінює структуру таблиці
- TRUNCATE викидає із таблиці всі дані
- DROP знищує таблицю (зазвичай — необоротно).

Data Manipulation Language

- INSERT додає рядки до таблиці, що існує.
- UPDATE модифікує рядки, що вже існують.
- DELETE видаляє рядки, що вже існують, з таблиці.
- та ніші (наприклад, MERGE)

INSERT

```
INSERT INTO students
 (first_name, last_name, average_score, homeworks)
 VALUES
 ('Кравець', 'Іван', 11.5, 22);

INSERT INTO students
 (first_name, last_name, average_score, homeworks)
 VALUES
 ('Петров', 'Іван', 5.5, 12),
 ('Іванов', 'Петро', 7.8, 21),
 ('Диванова', 'Зульфія', 12.0, 40);
```

UPDATE, DELETE

```
UPDATE students
 SET dateofbirth = '1990-11-15'
 WHERE first_name = 'Кравець' &&  last_name = 'Іван';
```

```
DELETE FROM students
 WHERE first_name = 'Кравець' &&  last_name = 'Іван';
```

SELECT

Можна сказати, найважливіша команда — отримання даних. Має багато умов та опцій, зокрема:

- FROM — з якої таблиці отримувати дані,
- WHERE — умова, якій повинні задовольняти вибрані рядки
- ORDER BY — за яким стовпцем сортувати, і як — за спаданням, чи зростанням. Наприклад, щоб вибрати
- LIMIT — які рядки із результату, повертати, (наприклад, діапазон: LIMIT 0, 20 — з нульового, 20 рядків).
- GROUP BY — групування згідно якогось стовпця,
- HAVING — дозволяє обмежувати результат виконання агрегатних функцій,
- Для вказання "вибрати всі", слід використати '*'.

```
SELECT *
    FROM students;

SELECT *
    FROM students
    LIMIT 1, 2;

SELECT first_name, dateofbirth
    FROM students;
```

```
SELECT SUM( homeworks )
    FROM students;

SELECT first_name, SUM(homeworks) AS FamilyHomeworks
    FROM students
    GROUP BY first_name;

SELECT first_name, SUM(homeworks) AS FamilyHomeworks
    FROM students
    GROUP BY first_name
    HAVING FamilyHomeworks > 5;

SELECT *
    FROM students
    ORDER BY average_score;

SELECT *
    FROM students
    ORDER BY average_score ASC;

SELECT *
    FROM students
    ORDER BY average_score DESC ;
```

SELECT ... WHERE

```
SELECT *
    FROM students
    WHERE first_name = 'Кравець';

SELECT *
    FROM students
    WHERE average_score > 10;

SELECT first_name, last_name
    FROM students
    WHERE average_score < 10 AND homeworks > 10;

SELECT first_name, last_name
    FROM students
    WHERE average_score BETWEEN 5 AND 10 ;
```

Взагалі, умова може включати предикати AND, OR, NOT, LIKE (відповідає заданому шаблону), BETWEEN, IS (не є NULL), IS NOT, IN (одне із кількох можливих значень), IS NOT DISTINCT FROM (обидва рівні, або обидва NULL) ключове слово NULL, оператори відношення та рівності (<, >, <>, =, >=, <=).

Ще приклад із вікіпедії — об'єднання даних із різних таблиць:

```
SELECT Book.title AS Title,
       COUNT(*) AS Authors
 FROM  Book
 JOIN  Book_author
   ON  Book.isbn = Book_author.isbn
 GROUP BY Book.title;
```

Subqueries

Аргументами запитів можуть бути вкладені запити, наприклад — вибірка студентів, із результатом, меншим за середній:

```
SELECT first_name, last_name
    FROM students
    WHERE average_score
            < (SELECT AVG(average_score) FROM students);
```

TCL (Transaction Control Language) — робота з транзакціями[3].

- Початок транзакції та кінець транзакції обгортають оператори маніпуляції даними.
- START TRANSACTION (чи BEGIN WORK, або BEGIN TRANSACTION , в залежності від діалекту), починає транзакцію,
- SAVE TRANSACTION (SAVEPOINT) — зберігає стан бази даних на цей момент, даючи можливість потім відновити його, за допомогою ROLLBACK.
- ROLLBACK — відміняє всі зміни з часів останнього COMMIT або SAVEPOINT.
- COMMIT — прийняти зміни до бази, зробити їх постійними.

START **TRANSACTION**;
UPDATE Account **SET** amount=amount−200 **WHERE** account_number=12
UPDATE Account **SET** amount=amount+200 **WHERE** account_number=23

IF ERRORS=0 **COMMIT**;
IF ERRORS<>0 **ROLLBACK**;

[3] За мотивами вікіпедії.

DCL (Data Control Language) — робота з правами[4]

- GRANT — надає користувачам права виконувати певні операції.
- REVOKE — забирає надані раніше права (включаючи права по замовчуванню).

```
GRANT SELECT, UPDATE
 ON example
 TO some_user, another_user;
```

```
REVOKE SELECT, UPDATE
 ON example
 FROM some_user, another_user;
```

[4] За мотивами вікіпедії

Типи даних

Деякі типи даних.

Текстові стрічки:

- CHARACTER(n), CHAR(n) — стрічка, розміром n символі, вирівняна пробілами до потрібної довжини,
- CHARACTER VARYING(n), VARCHAR(n) — стрічки змінного розміру, із максимальним розміром n.

Стрічки бітів:

- BIT(n), BIT VARYING(n) — стрічка бітів, рівно n чи до n, розміром, відповідно.

Числа:

- INTEGER, SMALLINT і BIGINT
- FLOAT, REAL і DOUBLE PRECISION
- NUMERIC(precision, scale) або DECIMAL(precision, scale)

Дані та час

- DATE: для даних (наприклад 2011-05-03)
- TIME: для часу (15:51:36). Квантом, зазвичай, є тік (tick) — 100 наносекунд.
- TIME WITH TIME ZONE або TIMETZ — те ж, що і TIME, але із включенням інформації про часовий пояс.
- TIMESTAMP — DATE і TIME разом (2011-05-03 15:51:36).
- TIMESTAMP WITH TIME ZONE або TIMESTAMPTZ — те ж, що і TIMESTAMP, але із включенням інформації про часовий пояс.
- Реалізація, зазвичай, включає функції для перетворення стрічки в дату (`TO_DATE`, `TO_TIME`, `TO_TIMESTAMP`) та звертання до окремих частин часу – секунд, хвилин, тощо. Є функція отримання поточного часу — NOW.

Увага: не всі реалізації підтримують всі наведені типи. Зокрема, sqlite підтримує лише класи типів (Storage Classes) NULL, INTEGER, REAL, TEXT, BLOB, (кожен із яких може містити різні конкретні типи, наприклад TINYINT, SMALLINT, INT, тощо, в INTEGER)а для маніпуляції, скажімо, датами, використовуються його функції.

Тернарна логіка

- SQL використовує логіку із трьома можливими значеннями: **True**, **False**, **Unknown** (NULL).

AND	True	False	Unknown
True	True	False	Unknown
False	False	False	False
Unknown	Unknown	False	Unknown

OR	True	False	Unknown
True	True	True	True
False	True	False	Unknown
Unknown	True	Unknown	Unknown

=	True	False	Unknown
True	True	False	Unknown
False	False	True	Unknown
Unknown	Unknown	Unknown	Unknown

NOT	
True	False
False	True
Unknown	Unknown

Вбудовані (predefined) функції

Поділяються на два типи:

- Агреговані (aggregate functions) — працюють над множиною рядків, тобто, отримують одне значення від кожного рядка у множині, і повертають одне значення для цілої множини. Якщо їх вжито в GROUP BY, викликаються для кожної групи, інакше один раз для всіх рядків.
- Скалярні (scalar functions) — для кожного рядка отримують одне значення і повертають одне значення.

Агреговані функції

Всі функції викидають із розгляду рядки, що у досліджуваних стовпцях містять NULL (природно, це не стосується COUNT(*)).

- COUNT(*) — кількість рядків,
- COUNT(<column_name>) — кількість рядків, у яких стовпець column_name містить значення.
- MIN(<column_name>) — Мінімальне значення (лексикографічно для стрічок).
- MAX(<column_name>) — Мінімальне значення (лексикографічно для стрічок).
- SUM(<column_name>) — Сума всіх значень.
- AVG(<column_name>) — Середнє арифметичне всіх значень.
- Функції можуть використовуватися із модифікаторами ALL або DISTINCT. ALL по замовчуванню, DISTINCT означає, що використовуватимуться лише відмінні між собою значення – працюватиме лише із унікальними значеннями.

Не використовуйте агреговані функції в SELECT разом із звичайними стовпцями! Наступний код помилковий!

```
SELECT lastname, SUM(weight)
    FROM   person;
```

Детальніше див., наприклад, http://en.wikibooks.org/wiki/Structured_Query_Language/SELECT

Скалярні функції

Стрічкові функції:

- SUBSTRING(<colname> FROM <pos> FOR <len>) — повертає стрічку, починаючи з позиції pos (рахуючи від 1!), довжиною len.
- UPPER(<colname>) — повертає стрічку у верхньому регістрі.
- LOWER(<colname>) — повертає стрічку у нижньому регістрі.
- CHARACTER_LENGTH(<colname>) — повертає довжину стрічки
- TRIM(<colname>) — відкидає пробіли на початку і в кінці стрічки.
- TRIM(LEADING FROM <colname>) — відкидає пробіли на початку.
- TRIM(TRAILING FROM <colname>) — відкидає пробіли в кінці

Математичні функції:

- SQRT(<colname>), ABS(<colname>), MOD(<colname>, <divisor>) — остача від ділення значення комірки на divisor, FLOOR, CEIL, POWER, EXP, LN.

Також існує набір функцій для роботи із часом та датою.

Детальніше див., наприклад:
`http://en.wikibooks.org/wiki/SQL_Dialects_Reference`

"Перетворення" нашої бази на реляційну

- Нехай студентам потрібно ще вказувати групу. Нехай групи беруться із окремої таблиці.
- Створюємо таблицю груп

```
CREATE TABLE student_groups(
    id INTEGER PRIMARY KEY NOT NULL AUTO_INCREMENT ,
    student_group VARCHAR( 50 ) NOT NULL
);
```

- Додаємо поле до таблиці студентів:

```
ALTER TABLE students ADD group_id INTEGER ;
```

- Повідомляємо, що це поле буде зовнішнім ключем[5]:

```
ALTER TABLE students
    ADD FOREIGN KEY (group_id)
    REFERENCES student_groups (id);
```

- Заповнюємо даними

```
INSERT INTO student_groups
 (student_group) VALUES ('Група 11'), ('Група 22');

UPDATE students SET group_id=1 WHERE id=1 || id=3 ;
UPDATE students SET group_id=2 WHERE id=2 || id=4 ;
```

- http://sourceforge.net/projects/xampp/ — дистрибутив XAMPP: Apache, MySQL, PHP, Perl.
- http://uk.wikipedia.org/wiki/MySQL
- http://uk.wikipedia.org/wiki/Sqlite
- http://uk.wikipedia.org/wiki/SQL і посилання там.
- http://en.wikipedia.org/wiki/SQL і посилання там.
- Вікікнижка http://en.wikibooks.org/wiki/Structured_Query_Language.
- http://sql-help.ru/ — простенький підручник із великою кількістю прикладів.
- Реляційні бази із MySQL/phpMyAdmin: http://fellowtuts.com/php/setting-up-foreign-key-in-phpmyadmin/, https://learnable.com/books/php-mysql-novice-to-ninja-5th-edition/preview/foreign-keys-614c328

- Для роботи із БД в Qt існує окремий модуль, sql:

  ```
  QT += sql
  ```

- Відповідні класи оголошені в файлі заголовків <QtSql>.
- Складається із трьох рівнів.

Рівні роботи із БД в Qt

- Рівень драйверів. Класи цього рівня безпосередньо працюють із фізичними даними: QSqlDriver, QSqlDriverCreator, QSqlDriverCreatorBase, QSqlDriverPlugin, QSqlResult. Якщо немає потреби писати свій драйвер БД, перший рівень потреби використовувати немає.
- Програмний рівень — SQL API. Інтерфейс доступу до БД. Під'єднання до бази здійснюється класом QSqlDatabase, запити відсилаються за допомогою QSqlQuery. Іншими класами цього рівня є класи QSqlError, QSqlField, QSqlIndex, QSqlRecord.
- Рівень користувацького інтерфейсу. Надає дані віджетам, які здатні його прийняти. Тобто, моделі бази даних для роботи із представленнями: QSqlQueryModel, QSqlTableModel, QSqlRelationalTableModel.

Драйвери

Весь код, що залежить від конкретних баз даних, знаходиться в драйверах. Самим Qt підтримуються (хоч і не всі в Open Source версії) наступні системи керування базами даних:

Ім'я драйвера	DBMS
QDB2	IBM DB2 (версія 7.1 і новіші)
QIBASE	Borland InterBase
QMYSQL	MySQL
QOCI	Oracle Call Interface Driver
QODBC	Open Database Connectivity (ODBC) - Microsoft SQL Server
QPSQL	PostgreSQL (версія 7.3 і новіші)
QSQLITE2	SQLite version 2
QSQLITE	SQLite version 3
QTDS	Sybase Adaptive Server (застарілий, починаючи з Qt 4.7)

Всі бази вище, крім SQLite — клієнт-серверні. Ми працюватимемо із MySQL та SQLite.

QSqlDatabase

- Щоб під'єднатися до бази, слід використати статичний методі QSqlDatabase::addDatabase():

  QSqlDatabase db;
  .....
  db = QSqlDatabase::addDatabase(driverName);

  Так створюється з'єднання по замовчуванню – глобальне для програми.
- Також можна створити іменоване з'єднання – на випадок, коли треба працювати із кількома базами одночасно:

  db2 = QSqlDatabase::addDatabase(driverName, "SecondConnection");
- При повторних викликах для того ж імені (або без імені), попереднє під'єднання від'єднується.
- Перевірити валідність поточного драйвера можна методом isValid():

  bool QSqlDatabase::isValid() **const**

QSqlDatabase — керування налаштуваннями БД

- Наступні методи дозволяють налаштувати QSqlDatabase, встановивши інформацію про базу[6]:

```
db.setDatabaseName( myDbName );
db.setHostName( dbHost );
db.setUserName( userString );
db.setPassword( passWordString );
// db.setPort( 234 );
```

  Детальніше див. документацію (зокрема, зверніть увагу на QSqlDatabase::setConnectOptions).
- Відкривається база методом open(), який повертає true, якщо відкрити вдалося успішно, false в іншому випадку:

```
bool ok = db.open();
```

  Перевірити стан можна методами isOpen() та isOpenError().
- Якщо сталася помилка, взнати її можна за допомогою методу QSqlDatabase::lastError(), який повертає об'єкт класу QSqlError. (Детальніше про нього на наступному слайді.)
- Закрити з'єднання, повернувши всі ресурси та інвалідувавши всі запити, можна методом close():

```
void QSqlDatabase::close();
```

QSqlError

- QSqlError зберігає інформацію про помилки SQL баз/систем керування базами даних.
- Метод isValid() повертає true, якщо помилку було встановлено, false в іншому випадку.
- Цей клас має методи для отримання тексту помилки: databaseText(), що повертає повідомлення про помилку від бази; driverText() — текст від драйвера бази в Qt; та text(), який повертає їх об'єднання.
- Також можна отримати тип помилки:

  ```
  ErrorType QSqlError::type();
  ```

  Де ErrorType може приймати значення: QSqlError::NoError = 0, QSqlError::ConnectionError = 1, QSqlError::StatementError = 2, QSqlError::TransactionError = 3, QSqlError::UnknownError = 4.
- Та системний (залежний від бази) код помилки:

  ```
  QString QSqlError::nativeErrorCode() const;
  ```

- Таким чином, текст помилки можна отримати, наприклад, так:

  ```
  QString errMsg = QString("Error: %1").arg( db.lastError().text() );
  ```

QSqlDatabase — інформація про базу

- Метод tables() повертає список таблиць у базі:

  QStringList QSqlDatabase::tables(QSql::TableType type = QSql::Tables) **const**;

  QSqlRecord QSqlDatabase::record(**const** QString & tablename) **const**;

- Повертає QSqlRecord, заповнений списком всіх полів вказаної таблиці. Порядок полів – не фіксується!
- QSqlIndex QSqlDatabase::primaryIndex(**const** QString & tablename) **const**

  повертає головний (primary) індекс вказаної таблиці.

QSqlDatabase — транзакції

Керування транзакціями:

- Розпочати транзакцію, якщо вони підтримуються драйвером:

  bool QSqlDatabase::transaction()

- Прийняти (commit) транзакцію:

  bool QSqlDatabase::commit()

- Відхилити транзакцію:

  bool QSqlDatabase::rollback()

- Всі ці методи повертають true, якщо операцію вдалося здійснити.

Можливості баз даних

- Перевірити, які можливості реалізовує конкретний драйвер можна за допомогою:

  bool QSqlDriver::hasFeature(DriverFeature feature) **const**;

- Драйвер конкретної бази можна отримати за допомогою:

  QSqlDriver * QSqlDatabase::driver () **const**

  (Ім'я драйвера повертає метод QSqlDatabase::driverName(), а список доступних драйверів: QSqlDatabase::drivers()).

- enum DriverFeature містить можливості, що можуть підтримуватися, зокрема: QSqlDriver::Transactions = 0, QSqlDriver::QuerySize = 1, QSqlDriver::BLOB = 2, QSqlDriver::Unicode = 3, QSqlDriver::NamedPlaceholders = 5, QSqlDriver::PositionalPlaceholders = 6, QSqlDriver::CancelQuery = 14, і т.д.
- Таким чином, перевірити, чи база підтримує транзакції, можна так:

  db. driver ()->hasFeature(QSqlDriver::Transactions)

QSqlRecord та QSqlField

- QSqlRecord енкапсулює один запис (рядок) БД.
- Одному запису відповідає клас QSqlField.
- Встановити поле з номером чи іменем стовпця можна наступними методоми:

  void QSqlRecord::setValue(**int** index, **const** QVariant & val);
  void QSqlRecord::setValue(**const** QString & name, **const** QVariant & val);

- Їхні напарники value() дозволяють прочитати поле:

  QVariant QSqlRecord::value(**int** index);
  QVariant QSqlRecord::value(**const** QString & name);

- Зробити поле порожнім (null) можна аналогічною парою методів setNull().

- **int** QSqlRecord::indexOf(**const** QString & name) **const**;

  повертає індекс поля за його іменем, -1 якщо поле не знайдено.

- QString QSqlRecord::fieldName(**int** index) **const**;

  повертає ім'я поля за номером, порожню стрічку, якщо поле не знайдено.

QSqlRecord – 2

- **int** QSqlRecord::count() **const**;

  повертає кількість полів,

- а:

  bool QSqlRecord::isEmpty() **const**;

  чи взагалі тут щось є.

- Методи:

  void QSqlRecord::insert(**int** pos, **const** QSqlField & field);
  void QSqlRecord::replace(**int** pos, **const** QSqlField & field);
  void QSqlRecord::remove(**int** pos);

  відповідно, вставляють поле у вказаній позиції, заміняють або видаляють його.

- Також перевантажено оператори `operator!=`, `operator==`, `operator=`.

QSqlQuery — виконання команд SQL

- Клас QSqlQuery надає можливість виконувати SQL-запити та дозволяє переглядати їх результати.
- Найпростіший спосіб використання:

```
QSqlQuery query;
bool res = query.exec( "SELECT * FROM students;" );
```

- exec() повертає true, якщо запит виконано успішно, false в іншому випадку. Подивитися помилку SQL (на додачу чи на противагу помилці драйвера, можна за допомогою):

```
QSqlError QSqlQuery::lastError() const;
```

а метод lastQuery() поверне стрічку із останнім виконаним запитом.

- Конструктор по замовчуванню виконуватиме запити над глобальною базою (базою по замовчуванню) програми. Можна також вказати конкретну базу:

```
QSqlQuery::QSqlQuery(QSqlDatabase db)
```

або й запит, який слід зразу виконати:

```
QSqlQuery::QSqlQuery(const QString & query = QString(),
                     QSqlDatabase db = QSqlDatabase())
```

QSqlQuery — робота з результатами

- QSqlQuery після виконання поводиться як ітератор в стилі Java:

```
while ( query.next() ) {
    int idx=0;
    QString line1;
    while( query.value(idx).isValid() )
    {
        line1 += query.value(idx).toString() + "\t";
        ++idx;
    }
    ui->sqlResultText->append( line1 );
}
```

  Див. також at(), previous(), next(), first(), last(), seek().

- Методи value():

  QVariant QSqlQuery::value(**int** index) **const**;
  QVariant QSqlQuery::value(**const** QString & name) **const**;

  надають доступ до результату. Поля запиту виду `SELECT fld1, fld2, fld3 FROM ...` будуть йти у вказаному в запиті порядку. Однак, **УВАГА**, для запиту виду `SELECT * FROM ...` порядок **не фіксований!**

- Розмір результату можна взнати із використанням:

  ```
  int QSqlQuery::size() const;
  ```

  Поверне -1, якщо розмір результату запиту визначити неможливо, або база даних не підтримує такої можливості. Також повертає -1 для всіх запитів, які не є SELECT (перевірити можна методом isSelect()).
- Приклад із офіційної документації:
  `http://doc.qt.io/qt-5/sql-sqlstatements.html`

```
    QSqlQuery query; int numRows;
query.exec("SELECT name, salary FROM employee WHERE salary > 50000
    QSqlDatabase defaultDB = QSqlDatabase::database();
    if (defaultDB.driver()->hasFeature(QSqlDriver::QuerySize)) {
        numRows = query.size();
    } else {
        // this can be very slow
        query.last();
        numRows = query.at() + 1;
    }
```

- QSqlQuery::setForwardOnly(true) дозволяє перейти у режим руху тільки вперед (без можливості повертатися), який часто швидший і вимагає менше пам'яті.

Додавання записів у БД

- QSqlQuery може використовуватися і для модифікації бази — додавання, оновлення та знищення записів. Додавання варто розглянути окремо.
- Звичайно, можна робити так:

```
QSqlQuery query;
query.exec(" \
    INSERT INTO students\
     (first_name, last_name, average_score, homeworks)\
     VALUES\
     Кравець (", Іван ", 11.5, 22);      \
    ");
```

Однак, часто це незручно, особливо коли треба додавати багато однотипних записів.

Прив'язка (binding) – іменовані (named)

- Альтернативним способом є розділення побудови запиту та прив'язування даних до нього.
- Дані можна прив'язувати за іменем (named binding):

```
QSqlQuery query;
query.prepare("  \
    INSERT INTO students\
     (first_name, last_name, average_score, homeworks)\
     VALUES\
     (:first_name, :last_name, :average_score, :homeworks);    \
    ");
query.bindValue(":first_name", "Кравець");
query.bindValue(":last_name", "Іван");
query.bindValue(":average_score", "11.5");
query.bindValue(":homeworks", "22");
query.exec();
```

- Після prepare виконувати bindValue()/exec() можна стільки раз, скільки необхідно.
- Так як bindValue (і його альтернатив addBindValue), отримують QVariant, для передачі null слід використати порожній QVariant відповідного типу, наприклад QVariant(QVariant::String) для передачі null в поле типу стрічки.

Прив'язка (binding) – позиційні (positional)

- Альтернативний варіант — позиційні прив'язки (positional binding):

```
QSqlQuery query;
query.prepare(" \
    INSERT INTO students\
     (first_name, last_name, average_score, homeworks)\
     VALUES\
     (?, ?, ?, ?);      \
    ");
query.addBindValue("Кравець");
query.addBindValue("Іван");
query.addBindValue("11.5");
query.addBindValue("22");
query.exec();
```

- Якщо база даних підтримує використаний тип прив'язки, запит передається їй безпосередньо, інакше Qt емулює його. Реальний виконаний запит можна побачити за допомогою QSqlQuery::executedQuery().

Пакетний режим

- Метод:

  bool QSqlQuery::execBatch(BatchExecutionMode mode = ValuesAsRows)

  ще більше спрощує додавання багатьох значень. Він дозволяє виконати підготовлений запит для пакету даних.
- Приклад з офіційної документації (`http://doc.qt.io/qt-5/qsqlquery.html#execBatch`):

```
QSqlQuery q;
q.prepare("insert into myTable values (?, ?)");

QVariantList ints;
ints << 1 << 2 << 3 << 4;
q.addBindValue(ints);

QVariantList names;
names << "Harald" << "Boris" << "Trond" << QVariant(QVariant::String)
q.addBindValue(names);

if (!q.execBatch())
    qDebug() << q.lastError();
```

Нюанси пакетного режиму

- Кількість елементів у всіх QVariantList повинна співпадати, а фактичний тип в межах одного такого списку повинен залишатися тим же!
- Фактично, розраховуйте тільки на режим ValuesAsRows, ValuesAsColumns – спеціалізований для Ораклівського драйвера.

Активні запити

- Якщо запит було виконано, але ще не завершено, він знаходиться в активному стані. Перевірити це можна за допомогою методу isActive().
- Зробити запит неактивним можна за допомогою методів finish() або clear(). Той же результат дає деструктор класу QSqlQuery.
- Якщо запит активний, особливо коли мова про SELECT, commit() та rollback() для деяких баз не працюватимуть.
- Перевірити, чи не є активним якраз запит типу SELECT, можна методом isSelect().

- На третьому рівні Qt надає три класи для роботи із БД як з моделями: QSqlQueryModel, QSqlTableModel, QSqlRelationalTableModel.
- QSqlQueryModel — модель тільки для читання, що представляє результати SQL-запиту, іншими словами – результат роботи QSqlQuery.
- QSqlTableModel — модель для читання та запису однієї таблиці БД.
- QSqlRelationalTableModel додає до QSqlTableModel підтримку зовнішніх ключів, дозволяючи відображати, у зручній для користувача формі, дані із кількох таблиць реляційної БД.

QSqlQueryModel — 1

- QSqlQueryModel — модель тільки для читання, що представляє результати SQL-запиту, іншими словами – результат роботи QSqlQuery.
- Використовується, наприклад, так:

```
QSqlQueryModel model;
model.setQuery( ui->sqlRequestEdit2->toPlainText() );
```

- SQL-запит можна задати setQuery():

```
void    setQuery(const QSqlQuery & query);
void    setQuery(const QString & query,
                 const QSqlDatabase & db = QSqlDatabase());
```

Друге перевантаження дозволяє вказати базу, із якою має працювати запит.

- QSqlRecord QSqlQueryModel::record(**int** row) **const**

повертає рядок результат запиту, із номером row.

- QSqlRecord QSqlQueryModel::record() **const**

повертає порожній запис із інформацією про поля поточного запиту.

QSqlQueryModel — 2

- QSqlQueryModel є нащадком QAbstractTableModel, тому всі звичні "модельні" методи, такі як data() теж доступні.
- Очевидно, QSqlQueryModel вміє працювати із віджетами-представленнями, нащадками QAbstractItemView та QComboBox.
- Помилку запиту можна взнати звичною функцією:

```
QSqlError QSqlQueryModel::lastError() const;
```

- QSqlQueryModel — модель тільки для читання, однак, успадкувавши та перевизначивши setData() і flags(), можна дозволити змінювати дані. Див., також приклад тут: `http://doc.qt.io/qt-5/sql-presenting.html`. Але, в більшості випадків, краще скористатися QSqlTableModel.

QSqlTableModel — 1

- QSqlTableModel — модель для читання та запису однієї таблиці БД.
- Фактично, дозволяє працювати із БД, без безпосереднього використання SQL.
- Приклад використання:

```
QSqlTableModel model;
model.setTable( tableName );
model. setFilter ( filter );
model.setSort( colNumber, Qt::AscendingOrder);
model. select ();
```

- Для отримання даних із бази, та заповнення ними моделі, слід викликати select().
- Фільтр — стрічка, те, що стояло б після WHERE в SELECT, наприклад "`average_score>0`".

QSqlTableModel — 2

- Отримати доступ до рядка моделі можна за допомогою:

  QSqlRecord QSqlTableModel::record(**int** row) **const**;

- Змінити — його напарником:

  bool QSqlTableModel::setRecord(**int** row, **const** QSqlRecord & values);

- Також, звичайно, можна користуватися звичними засобами моделі — data()/setData() та іншими.
-

submit/submitAll

- Перевизначений в цій моделі submit() з QAbstractItemModel, відправляє дані, якщо стратегія редагування (**EditStrategy**) — OnRowChange або OnFieldChange. Для OnManualSubmit не робить нічого. Тобто, тоді зміни лише кешуються в моделі. Не оновлює модель, окрім зміненого рядка, із бази.
- Щоб відправити всі зміни в базу, в режимі OnManualSubmit, слід використати **submitAll()**. Якщо дані успішно відправлено, оновлює модель.
- revertAll() відміняє всі зміни, що очікують відправлення базі даних. (revertRow(int row) — те ж, для конкретного рядка)
- Модифіковані, але ще не відправлені, рядки відмічаються '*' у вертикальних заголовках. Аналогічно, видалені рядки, видалення яких ще не було наказано базі, залишаються, однак відмічаються знаком оклику: '!'.
- Стратегію редагування можна задати наступним методом:

```
void QSqlTableModel::setEditStrategy( EditStrategy  strategy );
```

QSqlRelationalTableModel — 1

- QSqlRelationalTableModel — додає до QSqlTableModel підтримку зовнішніх ключів, дозволяючи відображати, у зручній для користувача формі, дані із кількох таблиць реляційної БД.
- Крім вже розглянутих вище засобів QSqlTableModel, цей клас дозволяє керувати відношеннями між базами.
- Наприклад, у нашій базі є дві таблиці, students і student_groups. student_groups містить список груп (стовпець з іменем student_group) та ключ, id. students містить купу полів, зокрема, поле group_id, яке є зовнішнім ключем і вказує на student_groups. Тобто, для даного рядка, значення group_id вказує вибрати рядок із ключем id в таблиці student_groups та вважати, що поле цієї таблиці з іменем student_group і є назвою групи. Виразити таке відношення засобами QSqlRelationalTableModel можна так:

```
int groupsIdx = relationalModel->fieldIndex( "group_id" );
relationalModel->setRelation(groupsIdx ,
        QSqlRelation( "student_groups",
                      "id",
                      "student_group" ) );
```

QSqlRelationalTableModel — 2

- Прототип setRelation() наступний:

  void setRelation (**int** column, **const** QSqlRelation & relation);

  де column — стовпець головної таблиці, що містить зовнішній ключ. Його можна знайти за іменем з допомогою методу fieldIndex(), як в прикладі вище.
- QSqlRelation — допоміжний клас, що зберігає інформацію про зовнішній ключ. Його конструктор:

  QSqlRelation::QSqlRelation(**const** QString & tableName, **const** QString &

  приймає ім'я таблиці, якій відповідає зовнішній ключ (в нашому випадку – таблиці груп), власне сам зовнішній ключ (id) та поле, яке слід показувати користувачу, коли він запитує рядок із даним id (student_group).

Увага: з невідомих причин (помилка в моєму коді?, помилка в Qt?), QSqlRelationalTableModel відмовляється звертатися до підлеглих таблиць, відображаючи лише сам ключ (як робить QSqlTableModel), якщо не встановити хоч якийсь фільтр (навіть просто "1" — "підходять всі") методом setFilter.

QSqlRelationalDelegate

- Зручним доповненням до реляційної моделі є делегат QSqlRelationalDelegate, який дозволяє для зовнішніх полів вибирати елементи із списку підлеглої таблиці.

```
QTableView *view = new QTableView;
view->setModel(model);
view->setItemDelegate(new QSqlRelationalDelegate(view));
```

Нюанси QSqlRelationalTableModel

- Таблиця повинна містити головний ключ. При чому, цей ключ не повинен посилатися на інші таблиці. (”he table’s primary key may not contain a relation to another table.”)
- Якщо в таблиці є поля, якщо посилаються на рядки підлеглої таблиці, що не існують, вони не будуть фігурувати в моделі. Цілісність даних на совісті користувачів.
- Для setData() слід завжди використовувати роль Qt::EditRole, для data() — завжди Qt::DisplayRole.
- Детальніше див. `http://doc.qt.io/qt-5/qsqlrelationaltablemodel.html`, розділ notes.

QDataWidgetMapper

- Клас, що дозволяє відображати вміст моделі на віджети, можна дуже ефективно використовувати і з базами даних.
- Приклад використання — одна частина вікна це таблиця, а друга – детальна інформація щодо вибраного рядка.
- Відображаються так звані секції моделі, скажімо, різні віджети можуть відповідати різним стовпцям одного рядка (або навпаки).
- Приклад:

```
QDataWidgetMapper *mapper = new QDataWidgetMapper;
mapper->setModel(model);
mapper->addMapping(firstNameEdit, 1);
mapper->addMapping(averageScoreSpinBox, 3);
mapper->addMapping(groupChooser, 6);
mapper->toFirst();
```

- toFirst() переходить до першого рядка моделі, вибираючи дані саме з нього. Є також toFirst(), toNext(), toPrevious(), toLast(), setCurrentIndex().
- Є два режими повернення даних назад в модель: AutoSubmit та ManualSubmit. Перший поміщає дані зразу після того, як віджет втратив фокус, другий - по виклику submit().

- `http://doc.qt.io/qt-5/sql-programming.html` — SQL Programming (з точки зору Qt).
- `http://doc.qt.io/qt-5/examples-sql.html` — приклади роботи з БД з-під Qt.
- `http://doc.qt.io/qt-5/sql-connecting.html` — Connecting to Databases.
- `http://doc.qt.io/qt-5/sql-driver.html` – SQL Database Drivers
- `http://doc.qt.io/qt-5/sql-types.html` — відповідність типів Qt типам популярних SQL-DB.

